ASSIGNMENT - 2

NAME: M. Rumana khan

REG NO: 192471025

COURSE CODE / NAME: CSA0610 - Design and Analysis Of Algorithms

---

Q1 – Sorting Under Different Data Conditions (6 marks)

1.1 Aim

To compare Selection Sort, Bubble Sort and Insertion Sort on different input arrangements and decide which one suits an order-processing system where most orders are already sorted.

1.2 Prediction before implementation

Algorithm	Prediction	Reason
Selection Sort	Similar comparison count for sorted, random and reverse inputs.	It scans the remaining unsorted portion to find the minimum on every pass, regardless of existing order.
Bubble Sort	Very efficient on already/nearly sorted input with an early-stop flag; more work on reverse order.	It stops if a full pass makes no swaps, but reverse data requires many adjacent swaps.
Insertion Sort	Best of these three for nearly sorted data; worst for reverse order.	It shifts only items that are out of place. Few inversions mean few shifts.

1.3 Test datasets

Condition	Input	Why it is useful
Nearly sorted	[1, 2, 3, 5, 4, 6, 7]	Only one neighbouring pair is out of order.
Significantly disordered	[7, 6, 5, 4, 3, 2, 1]	Reverse order creates many shifts/swaps.
Edge case: already sorted	[1, 2, 3, 4, 5, 6, 7]	Tests whether an algorithm benefits from existing order.

1.4 Implementation results

Input condition	Algorithm	Comparisons	Swaps	Moves
Nearly sorted	Selection Sort	21	1	3
Nearly sorted	Bubble Sort	11	1	3
Nearly sorted	Insertion Sort	7	0	7
Reverse order	Selection Sort	21	3	9
Reverse order	Bubble Sort	21	21	63
Reverse order	Insertion Sort	21	0	27
Already sorted (edge case)	Selection Sort	21	0	0
Already sorted (edge case)	Bubble Sort	6	0	0
Already sorted (edge case)	Insertion Sort	6	0	6

Counting convention: A comparison is counted whenever two data values are compared. A swap is an exchange of two positions. For Selection Sort and Bubble Sort, one swap is counted as three assignments/moves. In Insertion Sort, each shift and final key placement is counted as a move. Therefore, move counts are implementation-specific and should be compared using this same convention.

1.5 Interpretation

The results show that the input arrangement affects sorting performance. Selection Sort performs 21 comparisons for all three datasets because it searches the remaining unsorted portion on every pass. Bubble Sort performs only 6 comparisons on the already sorted list because it stops when no swaps occur. Insertion Sort performs only 7 comparisons on the nearly sorted list, making it suitable for small batches of mostly ordered data. Reverse-order input increases the number of shifts in Insertion Sort and swaps in Bubble Sort.

In simple words, Selection Sort performs the same number of value comparisons for all three seven-item datasets because its search for the minimum does not stop early. Bubble Sort makes one pass and stops on already sorted data, while Insertion Sort performs very few comparisons and shifts when the input is nearly sorted. Reverse order causes Insertion Sort to shift many earlier elements, and Bubble Sort performs many swaps.

1.6 Recommendation

I recommend Insertion Sort for this fulfilment centre's small batches of new orders because the brief says about 95% of the existing orders are already in the correct priority order. Insertion Sort can insert a few new or displaced values into an almost-sorted sequence with relatively little shifting. Bubble Sort with an early-stop flag can also detect a fully sorted list quickly, but Insertion Sort is generally a stronger fit for nearly sorted data. For very large lists, a production system should benchmark a library sorting implementation rather than rely on these simple quadratic algorithms.

1.7 Complexity

Selection Sort: $\Theta(n^2)$ comparisons in best, average and worst cases; $O(1)$ auxiliary space.

Bubble Sort with early stop: best case $\Theta(n)$, worst case $\Theta(n^2)$; $O(1)$ auxiliary space.

Insertion Sort: best case $\Theta(n)$, average/worst case $\Theta(n^2)$; $O(1)$ auxiliary space.

Q2 – When Is Sequential Search an Appropriate Engineering Choice? (4 marks)

2.1 Aim and method

Sequential Search checks records one at a time until the requested Tool ID is found or the list ends. The experiment uses sorted IDs only to make target positions easy to describe; Sequential Search itself does not require sorted data.

List size	Search condition	Target
10	First item	1
10	Middle item	6
10	Last item	10
10	Missing item	10
100	First item	1
100	Middle item	51
100	Last item	100
100	Missing item	100
1000	First item	1
1000	Middle item	501
1000	Last item	1000
1000	Missing item	1000

2.2 Analysis

- Successful search at position k requires k comparisons when counting the first record as position 1.
- An unsuccessful search checks all n records, so it needs n comparisons.
- For a uniformly likely successful search, the average is (n + 1) / 2 comparisons.
- Sequential Search is reasonable for small temporary lists, infrequent requests, or records that are already stored in a simple unsorted collection. It is easy to implement and needs no preprocessing.
- I would stop recommending it when the list becomes large and requests are frequent, because each lookup may inspect a substantial fraction of all records. With 20 facilities and many active tool records, an indexed hash table or database index would usually be a better choice.

2.3 Changed operating condition

Expansion from one hangar to 20 facilities increases both the number of records and the likely lookup frequency. The original recommendation therefore changes: sequential search may remain acceptable for a tiny local cache, but not as the main lookup method across all facilities. A hash index on Tool ID can provide expected near-constant-time lookups, while a database index can support concurrent updates and reliable tracking.

2.4 Complexity

Best case: O(1); average and worst case: O(n). Extra space for the search itself is O(1).

Q3 – Engineer the Behaviour of Brute-Force String Matching (6 marks)

3.1 Aim

To observe how the arrangement of characters in a text and a pattern affects the number of character comparisons in a straightforward pattern-matching algorithm.

3.2 Algorithm

- Align the pattern at each possible text position.
- Compare characters from left to right.
- Stop the current alignment at the first mismatch; report a match if every pattern character matches.
- Continue until all valid alignments have been tested.

3.3 Predictions and test data

Test	Text / pattern design	Prediction
Low work	Text: SYSTEM ALERT: disk online; Pattern: ALERT	A mismatch early in many alignments should stop comparisons quickly; the pattern also occurs once.
Repeated-prefix worst-style case	Text: 40 copies of 'a' followed by 'b'; Pattern:	Each alignment matches many 'a' characters and then

	19 copies of 'a' followed by 'c'	mismatches at the last character, causing repeated work.
Additional case	Text: header.....LOGIN_SUCCESS; Pattern: LOGIN_SUCCESS	The match occurs near the end; preceding alignments mostly fail, generally early.

Test	Text length	Pattern length	Match positions (0-based)	Character comparisons
Low work / early mismatch	25	5	[7]	25
High work / repeated prefix	41	20	[]	440
Exact match near end	25	13	[12]	25

3.4 Interpretation

The repeated-prefix case performs much more work because each new alignment rechecks characters that matched in the previous alignment. The text and pattern share a long prefix, so a mismatch near the end is expensive. By contrast, a mismatch at the first character ends an alignment quickly. This shows that input length alone does not fully describe the observed work: character arrangement and prefix overlap matter too.

3.5 Complexity

For text length n and pattern length m, brute-force string matching has a worst-case time complexity of $O((n - m + 1)m)$, often written $O(nm)$, and $O(1)$ auxiliary space apart from stored match positions.

Q4 – Closest Pair and the Cost of Examining Every Possibility (6 marks)

4.1 Algorithm design

- Examine each pair of points exactly once by choosing indices i and j where i < j.
- Compute squared Euclidean distance: $(x_i - x□)^2 + (y_i - y□)^2$.
- Keep the pair with the smallest squared distance. Taking a square root is needed only for the final reported distance, not to compare pairs.
- Using i < j avoids checking both (i, j) and (j, i), and avoids pairing a point with itself.

4.2 Mathematical derivation

For n drones, the first drone can pair with n − 1 others, the second with n − 2 new others, and so on. Thus the number of unique pairs is (n − 1) + (n − 2) + … + 1 = n(n − 1)/2 = n choose 2.

Number of drones n	Pairs examined	Distance calculations	Theoretical pair count	Runtime (seconds)
10	45	45	n(n-1)/2 = 45	0.0000336
100	4950	4950	n(n-1)/2 = 4950	0.0013253
1000	499500	499500	n(n-1)/2 = 499500	0.1488446

4.3 Scalability and response-time criterion

For this report, I define an acceptable response time for a prototype monitoring calculation as at most 1 second on the computer used for testing. Runtime should be measured locally because hardware and interpreter overhead differ. The number of pairs examined increases quadratically with the number of drones. For 10 drones, only 45 pairs are examined, whereas 1,000 drones require 499,500 pair checks. The measured runtime increases from approximately 0.000034 seconds to 0.149 seconds. Although the largest tested input remains below the report's one-second response-time target, further growth could make the brute-force approach unsuitable for frequent real-time monitoring.

4.4 Source of inefficiency and redesign direction

The cost comes from considering every possible pair, even when many pairs are obviously far apart. A redesigned strategy should exploit spatial ordering or spatial regions so it can rule out many pairs without calculating every distance. A spatial

grid or a divide-and-conquer approach can reduce the number of distance calculations, though each requires additional logic and possibly additional memory.

4.5 Complexity

Time: $\Theta(n^2)$. Auxiliary space: O(1) when only the best pair and count are stored.

Q5 – Derive a Convex Hull Algorithm from a Geometric Principle (7 marks)

5.1 Geometric decision rule

For two candidate points A and B, use the cross product $(B - A) \times (P - A)$ to determine which side of the directed line AB a third point P lies on. A positive result means one side, a negative result means the other side, and zero means collinear.

If all other points lie on one side of AB (or on the line), AB is a supporting line and is a candidate hull edge. If points occur on both sides, AB cannot be an outer hull edge. The algorithm checks candidate pairs and scans the other points to apply this test.

5.2 Pseudocode

```
BRUTE_FORCE_HULL(points):
  remove duplicate points
  hull_edges = empty set
  for each pair A, B with A before B:
    positive = false; negative = false
    for each point P other than A and B:
      s = cross(B - A, P - A)
      if s > 0: positive = true
      if s < 0: negative = true
      if positive and negative: stop checking this pair
    if not (positive and negative):
      record AB as a candidate supporting edge
  order the boundary vertices around the outside
  return the ordered hull
```

Implementation note: the code checks candidate supporting edges and uses a monotonic-chain ordering step to return a consistently ordered boundary for

visualisation. The edge-validation idea is brute force; the ordering step is included to make the result practical to plot.

5.3 Test configurations

Test configuration	Ordered hull vertices	Side tests
Boundary and interior	[(0, 0), (4, 0), (4, 3), (0, 3)]	77
Challenge	[(-2, 0), (0, -2), (2, 0), (0, 3)]	120

5.4 Visualisation guidance

The first plot should show a rectangular convex hull with the interior points remaining inside its boundary. The second plot should show a four-vertex hull formed by the leftmost, bottommost, rightmost and topmost points. Connecting the hull vertices in order and closing the boundary helps demonstrate how the algorithm identifies the outermost points while excluding interior points.

5.5 Correctness reasoning

A convex hull edge lies on a supporting line: the convex set of all points is entirely on one side of that line. If the candidate line has input points strictly on both sides, it cuts through the point set and cannot support the outer boundary. If all points are on one side or collinear, the line is consistent with a supporting boundary line. Testing every candidate pair therefore identifies supporting lines that can contain hull edges. Collinear points and duplicate points require care when producing the final ordered list.

5.6 Complexity and limitations

There are $O(n^2)$ candidate pairs and up to $O(n)$ point-side tests per pair, so the straightforward test has $O(n^3)$ worst-case time. Early stopping may reduce actual tests but does not improve the worst-case bound. The extra storage is $O(n)$ for points and the returned hull.

Q6 – Exhaustive Search and Combinatorial Growth (8 marks)

6.1 Strategy

- Represent a candidate combination as an n-bit mask: bit i is 1 if package i is selected and 0 otherwise.
- Generate every mask from 0 to $2^n - 1$, including the empty selection.
- For each mask, add selected package weights and values.
- Reject the candidate if total weight exceeds the payload limit.
- For feasible candidates, compare total value with the best value found so far and retain the better combination.

6.2 Small correctness-check instance

Package	Weight	Humanitarian value
P1	2	6
P2	3	8
P3	4	9
P4	5	10

Payload limit = 7. The exhaustive search returns the optimal selected packages and totals shown below.

Payload limit	Selected packages	Total weight	Total value	Subsets examined	Feasible subsets
7	P2, P3	7	17	16	9

This small case can be independently checked by listing the 16 possible subsets. The chosen set is feasible and has the maximum value among all subsets whose total weight is at most 7.

6.3 Growth prediction and experiment

Each of n packages can either be included or excluded. Therefore there are $2 \times 2 \times \ldots \times 2 = 2^n$ possible subsets. Adding one package doubles the number of candidate combinations.

Packages n	Candidate subsets examined	Feasible subsets in test
4	16	14
10	1024	867
15	32768	29182
20	1048576	964943

6.4 Deployment decision

For this educational prototype, I set a response-time budget of 1 second and would deploy exhaustive enumeration only for small instances after timing the code on the target machine. The number of candidates is 1,048,576 at n = 20, 33,554,432 at n = 25, and 1,073,741,824 at n = 30. Because each candidate also needs weight/value evaluation, growth soon becomes unacceptable. A conservative initial deployment boundary is $n \leq 20$ packages for this simple implementation, subject to local runtime validation.

6.5 Why growth becomes dramatic

The experiment demonstrates the exponential growth of exhaustive search. Increasing the number of packages from 10 to 20 increases the number of subsets from 1,024 to 1,048,576. Each additional package doubles the number of candidate subsets. Although the method can identify the optimal solution, it becomes increasingly expensive as the input grows. Dynamic programming or branch-and-bound may be more appropriate for larger instances.

6.6 Complexity

Time: $O(n \cdot 2^n)$ for the shown implementation because each subset may inspect n packages. Auxiliary space: O(n) for the input and chosen solution; the subset itself is represented by an integer mask.

Q7 – Diagnose, Redesign and Validate (13 marks)

Selected solution: Brute-force string matching

I selected Q3 because the original algorithm repeats comparisons of the same text characters when a long prefix matches and a later character fails. This is a clear bottleneck that can be addressed by reusing information about the pattern.

A. Diagnose the computational bottleneck (2 marks)

In the engineered test, the text contains 1,000 'a' characters followed by 'b', and the pattern contains 40 'a' characters followed by 'b'. At successive alignments, brute force rechecks long runs of 'a'. The main growing operation is character comparison; the original approach does not remember how much of the pattern has already matched. In general, worst-case comparisons grow as O(nm).

B. Define the design objective (2 marks)

Objective: reduce repeated text-pattern comparisons while returning exactly the same match positions. Measurable target: on the repeated-prefix test, perform fewer character comparisons than brute force, and preserve the match result.

C. Revised strategy (4 marks)

The redesign uses a prefix table (often called LPS: longest proper prefix that is also a suffix). Before scanning the text, it records how much of the pattern can still be reused after a mismatch. During matching, instead of moving the pattern back to its first character and rechecking text, the algorithm shifts the pattern state to the longest prefix that remains valid. This is the key property it exploits: pattern-prefix overlap.

- Preprocessing: build the LPS table in O(m) time.
- Search: advance through the text and pattern without moving the text index backwards.
- Repeated work avoided: matched text characters are not repeatedly compared against the same pattern prefix after a mismatch.
- Memory trade-off: O(m) extra memory for the LPS table.
- The preprocessing adds a small cost, so for tiny patterns or one-off short texts, brute force can remain simpler and may be competitive.

```
BUILD_LPS(pattern):
  lps[0] = 0
  length = 0
  i = 1
  while i < pattern_length:
    if pattern[i] == pattern[length]:
      length = length + 1
      lps[i] = length
      i = i + 1
    else if length > 0:
      length = lps[length - 1]
    else:
      lps[i] = 0
      i = i + 1
```

```
KMP_SEARCH(text, pattern):
  build lps
  i = 0; j = 0
  while i < text_length:
    compare text[i] with pattern[j]
    if equal:
      i = i + 1; j = j + 1
      if j == pattern_length:
        report i - j
        j = lps[j - 1]
    else if j > 0:
      j = lps[j - 1]
    else:
      i = i + 1
```

D. Prediction before testing (1 mark)

Prediction: the prefix-reuse design will return the same match positions and reduce the number of comparisons on repeated-prefix inputs. The main performance measure is total character comparisons, including prefix-table construction.

E. Implementation and validation (2 marks)

Method	Text length	Pattern length	Match positions	Comparisons counted
Brute force	1001	41	[960]	39401
Prefix reuse (KMP)	1001	41	[960]	2040

The count for KMP includes comparisons used during prefix-table construction and during the text scan. The two methods should report the same match positions. Run the appendix code to reproduce the counts in your environment.

F. Defend the final design (2 marks)

Both algorithms identify the match at position 960, confirming that the redesign preserves the result. Brute force performs 39,401 comparisons, whereas KMP performs 2,040 comparisons, including prefix-table construction. This is a reduction of approximately 94.82% in the counted comparisons. KMP is therefore more suitable for repeated-prefix inputs and larger text searches, although it requires additional preprocessing and memory.

Q7 overall conclusion

The redesign is based on a diagnosed cause—repeated prefix comparisons—rather than simply choosing a named algorithm because its complexity is better. The experiment checks both correctness (same match positions) and efficiency (comparison count).

Overall Conclusion

The experiments demonstrate that an algorithm should be selected according to both input characteristics and operating requirements. Insertion Sort suits nearly sorted batches; Sequential Search is acceptable for small and infrequently searched lists but weakens as records and request rates grow; brute-force string matching can repeat work on overlapping prefixes; brute-force closest pair examines $n(n-1)/2$ pairs; the brute-force convex-hull test can require cubic work; and exhaustive search explores 2^n subsets. For Q7, prefix reuse reduces repeated matching work by retaining information from previous comparisons.

Operation counts and theoretical growth provide more stable evidence than raw runtime alone. Runtime should still be measured on the target environment before making a deployment decision.

Appendix A – Complete Source Code

The following Python 3 source contains implementations for Q1–Q7 and prints core operation counts. Save it as maincode.py and run with Python 3. The code is kept in one file for convenient submission.

```
# Python 3 source for Q1–Q7
import math
import time
```

```
def sorting_algorithm(values, method):
    a = values[:]
    comparisons = swaps = moves = 0
    n = len(a)
    if method == "selection":
        for i in range(n - 1):
            minimum = i
            for j in range(i + 1, n):
                comparisons += 1
                if a[j] < a[minimum]:
                    minimum = j
            if minimum != i:
                a[i], a[minimum] = a[minimum], a[i]
                swaps += 1
                moves += 3
    elif method == "bubble":
        for i in range(n - 1):
            swapped = False
            for j in range(n - 1 - i):
                comparisons += 1
                if a[j] > a[j + 1]:
                    a[j], a[j + 1] = a[j + 1], a[j]
                    swaps += 1
                    moves += 3
                    swapped = True
            if not swapped:
                break
    elif method == "insertion":
        for i in range(1, n):
            key = a[i]
            j = i - 1
            while j >= 0:
                comparisons += 1
                if a[j] > key:
                    a[j + 1] = a[j]
                    moves += 1
                    j -= 1
```

```
            else:
                break
        a[j + 1] = key
        moves += 1
    else:
        raise ValueError("Unknown sorting method")
    return a, comparisons, swaps, moves
def sequential_search(records, target):
    comparisons = 0
    for index, value in enumerate(records):
        comparisons += 1
        if value == target:
            return index, comparisons
    return -1, comparisons
def brute_force_string_match(text, pattern):
    comparisons = 0
    matches = []
    if pattern == "":
        return list(range(len(text) + 1)), comparisons
    for start in range(len(text) - len(pattern) + 1):
        j = 0
        while j < len(pattern):
            comparisons += 1
            if text[start + j] != pattern[j]:
                break
            j += 1
        if j == len(pattern):
            matches.append(start)
    return matches, comparisons
def build_lps(pattern):
    lps = [0] * len(pattern)
    length = 0
    i = 1
    comparisons = 0
    while i < len(pattern):
        comparisons += 1
        if pattern[i] == pattern[length]:
```

```
            length += 1
            lps[i] = length
            i += 1
        elif length > 0:
            length = lps[length - 1]
        else:
            lps[i] = 0
            i += 1
    return lps, comparisons
def kmp_search(text, pattern):
    if pattern == "":
        return list(range(len(text) + 1)), 0
    lps, comparisons = build_lps(pattern)
    matches = []
    i = j = 0
    while i < len(text):
        comparisons += 1
        if text[i] == pattern[j]:
            i += 1
            j += 1
            if j == len(pattern):
                matches.append(i - j)
                j = lps[j - 1]
        elif j > 0:
            j = lps[j - 1]
        else:
            i += 1
    return matches, comparisons
def closest_pair(points):
    best_squared = float("inf")
    best_pair = None
    distance_calculations = 0
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            distance_calculations += 1
            dx = points[i][0] - points[j][0]
            dy = points[i][1] - points[j][1]
```

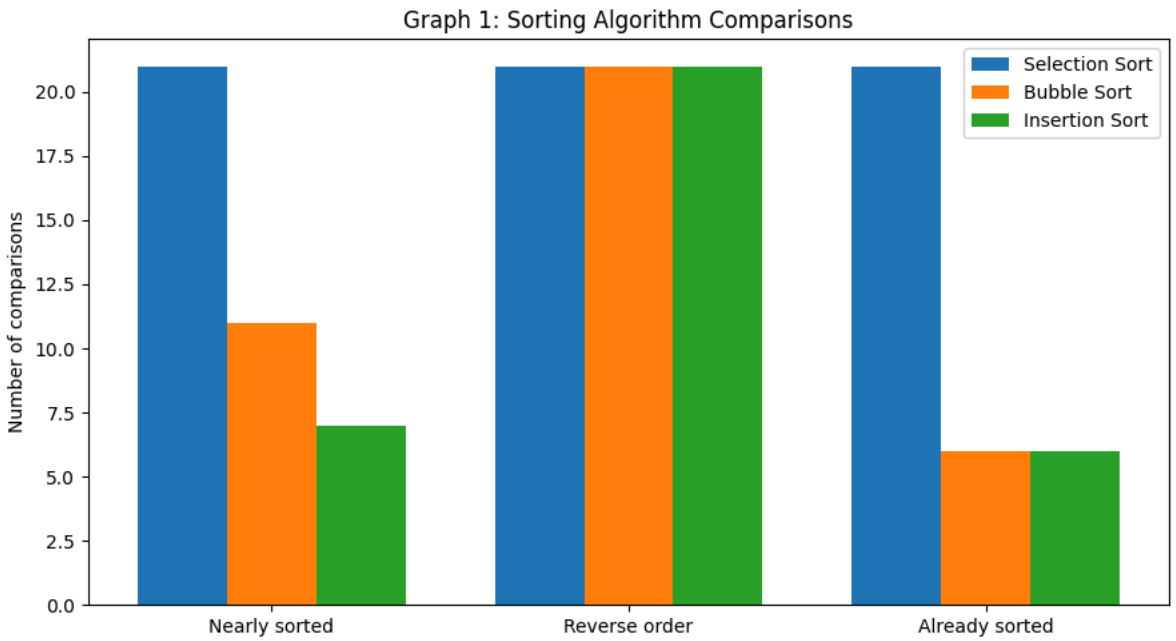
```
            squared = dx * dx + dy * dy
            if squared < best_squared:
                best_squared = squared
                best_pair = (points[i], points[j])
    distance = math.sqrt(best_squared) if best_pair else None
    return best_pair, distance, distance_calculations
def cross(o, a, b):
    return ((a[0] - o[0]) * (b[1] - o[1])
            - (a[1] - o[1]) * (b[0] - o[0]))
def convex_hull(points):
    # Brute-force supporting-line test, then order vertices for output.
    pts = sorted(set(points))
    n = len(pts)
    side_tests = 0
    if n <= 1:
        return pts, side_tests
    candidate_edges = []
    for i in range(n):
        for j in range(i + 1, n):
            positive = negative = False
            for k in range(n):
                if k == i or k == j:
                    continue
                side_tests += 1
                value = cross(pts[i], pts[j], pts[k])
                if value > 0:
                    positive = True
                elif value < 0:
                    negative = True
                if positive and negative:
                    break
            if not (positive and negative):
                candidate_edges.append((pts[i], pts[j]))
    # Order the boundary vertices for display.
    lower = []
    for point in pts:
        while len(lower) >= 2 and cross(lower[-2], lower[-1], point) <= 0:
```

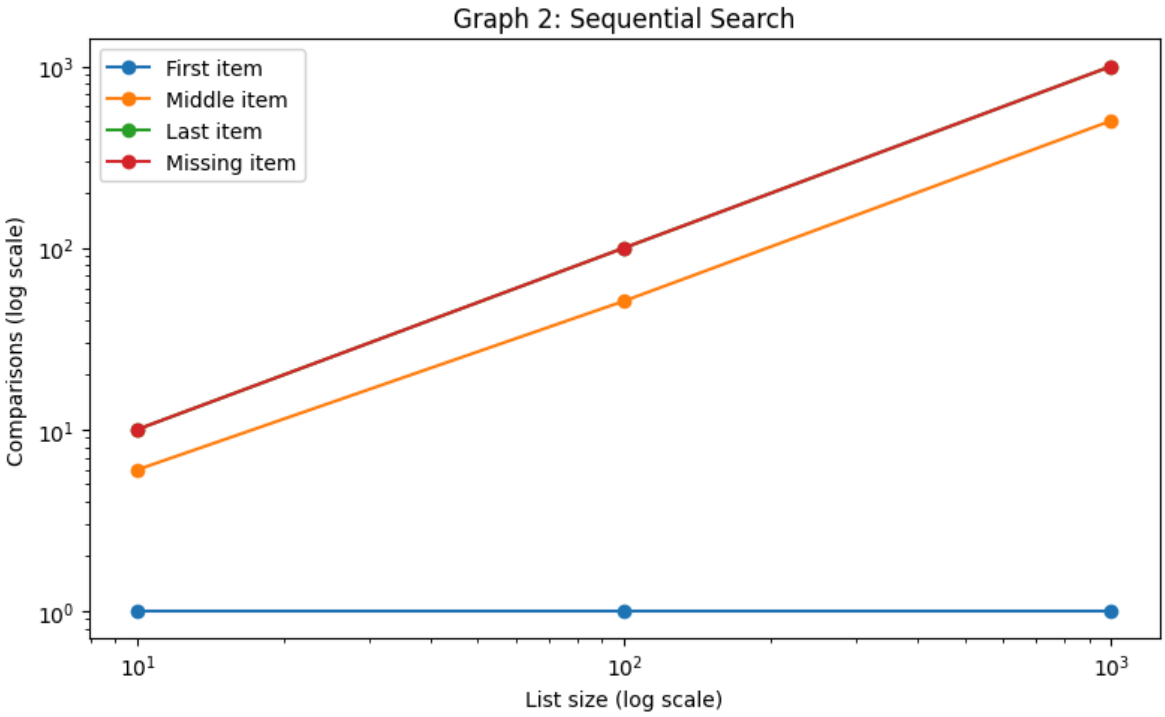
```
            lower.pop()
        lower.append(point)
    upper = []
    for point in reversed(pts):
        while len(upper) >= 2 and cross(upper[-2], upper[-1], point) <= 0:
            upper.pop()
        upper.append(point)
    ordered_hull = lower[:-1] + upper[:-1]
    return ordered_hull, side_tests
def exhaustive_knapsack(weights, values, capacity):
    n = len(weights)
    subsets_examined = 0
    feasible_subsets = 0
    best_value = -1
    best_weight = 0
    best_mask = 0
    for mask in range(1 << n):
        subsets_examined += 1
        total_weight = total_value = 0
        for i in range(n):
            if mask & (1 << i):
                total_weight += weights[i]
                total_value += values[i]
        if total_weight <= capacity:
            feasible_subsets += 1
            if total_value > best_value:
                best_value = total_value
                best_weight = total_weight
                best_mask = mask
    selected = [i for i in range(n) if best_mask & (1 << i)]
    return selected, best_weight, best_value, subsets_examined, feasible_subsets
def main():
    print("Q1: SORTING")
    datasets = {
        "Nearly sorted": [1, 2, 3, 5, 4, 6, 7],
        "Reverse order": [7, 6, 5, 4, 3, 2, 1],
        "Already sorted": [1, 2, 3, 4, 5, 6, 7],
```

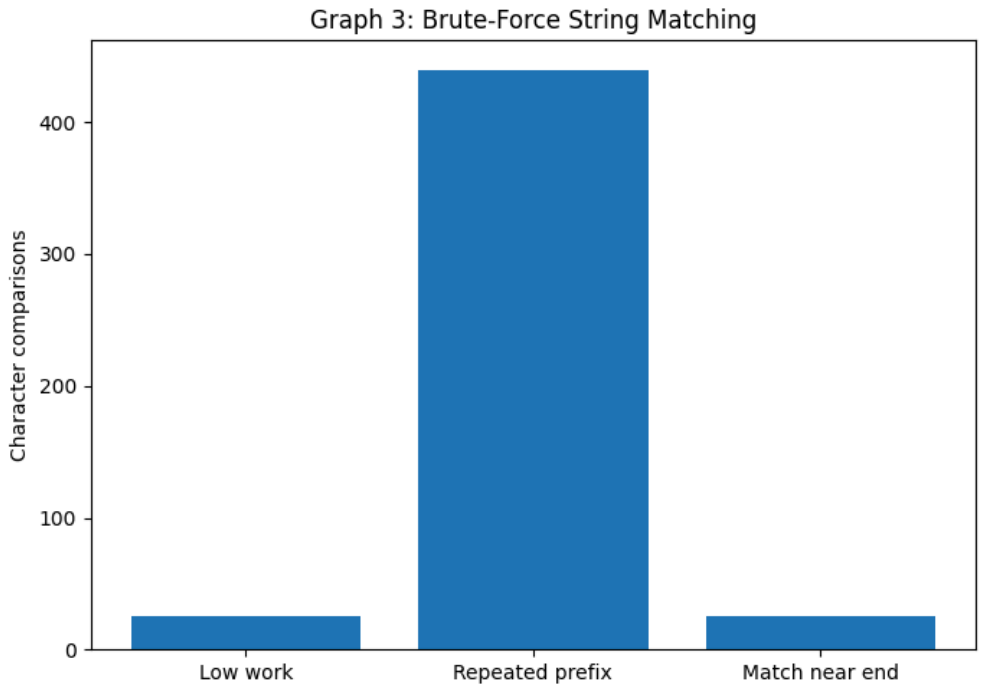
```
}
for label, data in datasets.items():
    for method in ("selection", "bubble", "insertion"):
        start = time.perf_counter()
        result, comps, swaps, moves = sorting_algorithm(data, method)
        elapsed = time.perf_counter() - start
        print(label, method, "sorted=", result, "comparisons=", comps,
              "swaps=", swaps, "moves=", moves, "seconds=", elapsed)
print("\\nQ2: SEQUENTIAL SEARCH")
for n in (10, 100, 1000):
    records = list(range(1, n + 1))
    for label, target in (("first", 1), ("middle", records[n // 2]),
                          ("last", n), ("missing", n + 1)):
        _, comps = sequential_search(records, target)
        print("n=", n, label, "comparisons=", comps)
print("\\nQ3 and Q7: STRING MATCHING")
tests = [
    ("low-work", "SYSTEM ALERT: disk online", "ALERT"),
    ("repeated-prefix", "a" * 40 + "b", "a" * 19 + "c"),
    ("match-near-end", "header.....LOGIN_SUCCESS", "LOGIN_SUCCESS"),
]
for label, text, pattern in tests:
    matches, comps = brute_force_string_match(text, pattern)
    print(label, "matches=", matches, "comparisons=", comps)
text = "a" * 1000 + "b"
pattern = "a" * 40 + "b"
start = time.perf_counter()
brute_matches, brute_comps = brute_force_string_match(text, pattern)
brute_time = time.perf_counter() - start
start = time.perf_counter()
kmp_matches, kmp_comps = kmp_search(text, pattern)
kmp_time = time.perf_counter() - start
print("Q7 brute force:", brute_matches, brute_comps, brute_time)
print("Q7 prefix reuse:", kmp_matches, kmp_comps, kmp_time)
print("\\nQ4: CLOSEST PAIR")
for n in (10, 100, 1000):
    points = [(i, (i * 37) % 101) for i in range(n)]
```

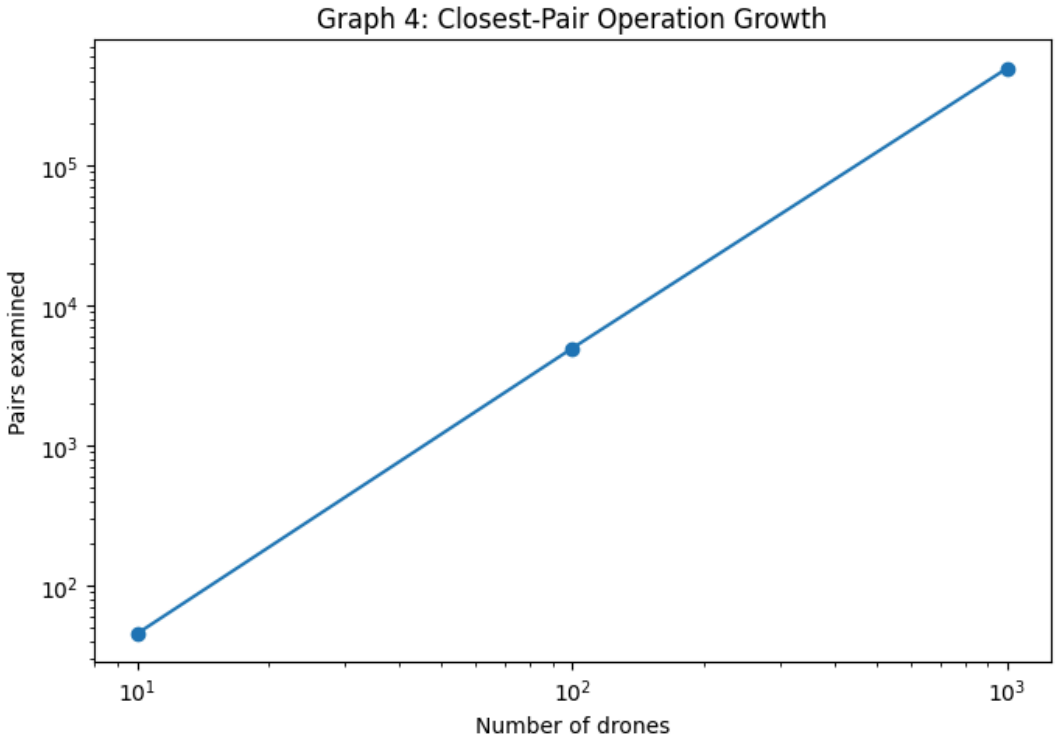
```
        start = time.perf_counter()
        pair, distance, count = closest_pair(points)
        elapsed = time.perf_counter() - start
        print("n=", n, "pairs=", count, "distance_calculations=", count,
              "closest_pair=", pair, "distance=", distance, "seconds=", elapsed)
    print("\\nQ5: CONVEX HULL")
    point_sets = [
        ("boundary_and_interior",
         [(0, 0), (4, 0), (4, 3), (0, 3), (2, 1), (1, 2), (3, 2)]),
        ("challenge",
         [(-2, 0), (-1, 0), (0, 0), (1, 0), (2, 0), (0, 3), (0, -2), (1, 1)]),
    ]
    for label, points in point_sets:
        hull, tests = convex_hull(points)
        print(label, "hull=", hull, "side_tests=", tests)
    print("\\nQ6: EXHAUSTIVE KNAPSACK")
    items = [("P1", 2, 6), ("P2", 3, 8), ("P3", 4, 9), ("P4", 5, 10)]
    weights = [item[1] for item in items]
    values = [item[2] for item in items]
    chosen, weight, value, examined, feasible = exhaustive_knapsack(weights, values, 7)
    print("small instance selected=", [items[i][0] for i in chosen],
          "weight=", weight, "value=", value,
          "subsets_examined=", examined, "feasible=", feasible)
    for n in (4, 10, 15, 20):
        weights = [(i % 5) + 1 for i in range(n)]
        values = [((i * 7) % 13) + 3 for i in range(n)]
        _, _, _, examined, feasible = exhaustive_knapsack(weights, values, max(5, n * 2))
        print("n=", n, "subsets=", examined, "feasible=", feasible)
if __name__ == "__main__":
    main()
```

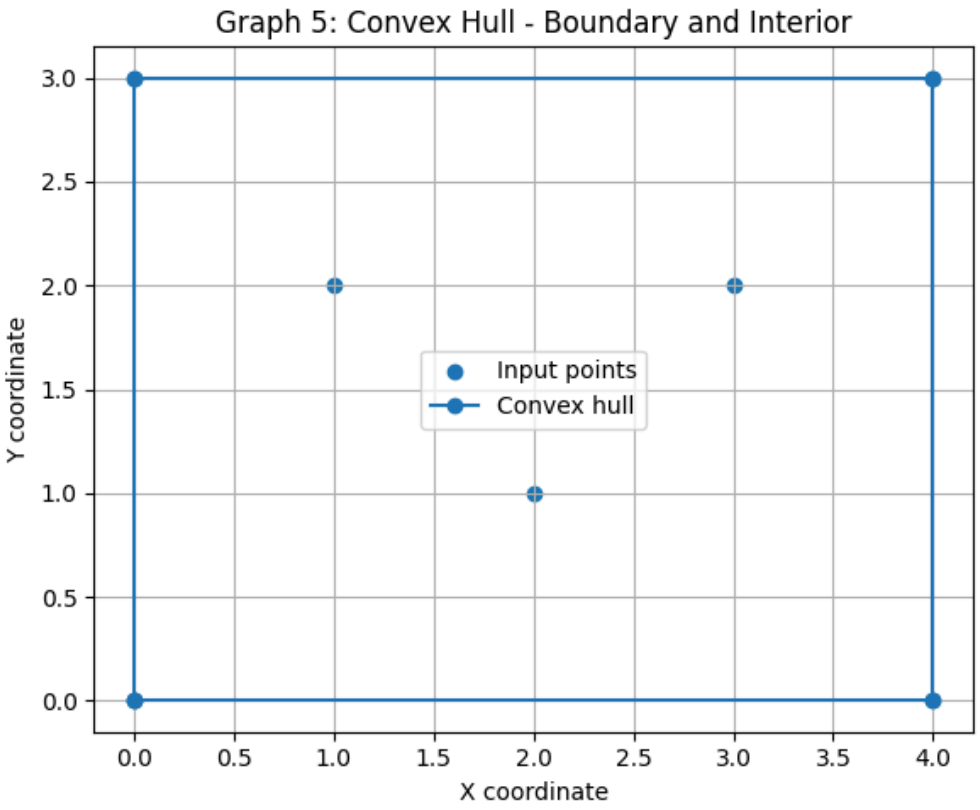
Appendix B – Visual Evidence

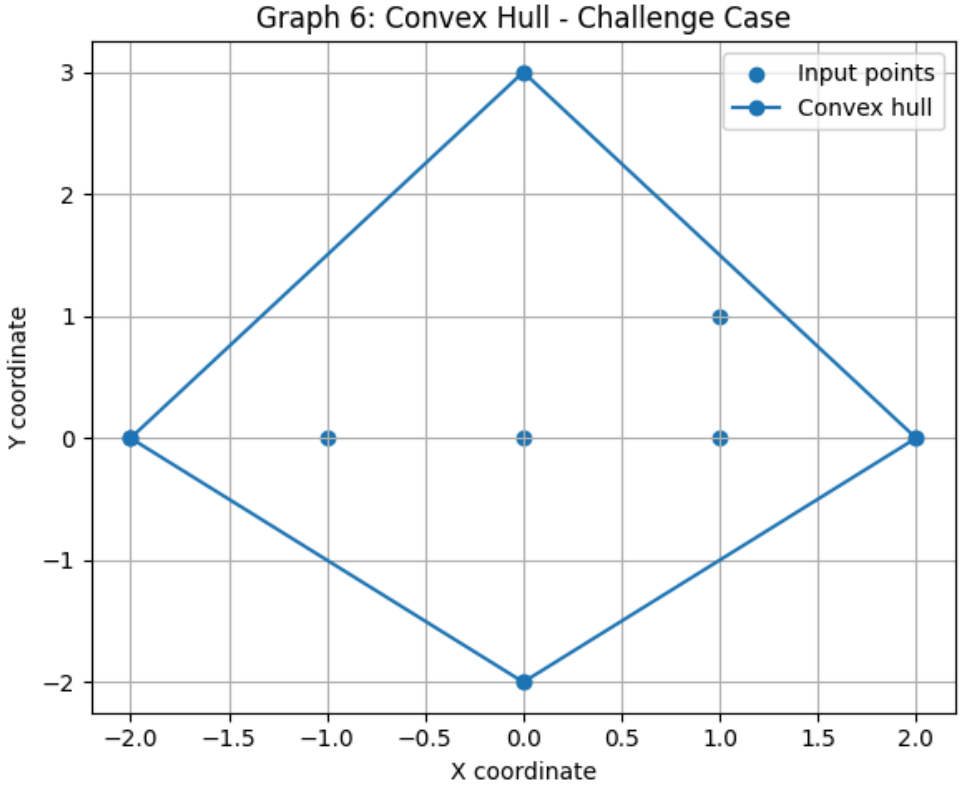

Graph 1: Sorting Algorithm Comparisons
Selection Sort
Bubble Sort
Insertion Sort
Number of comparisons
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
Nearly sorted
Reverse order
Already sorted

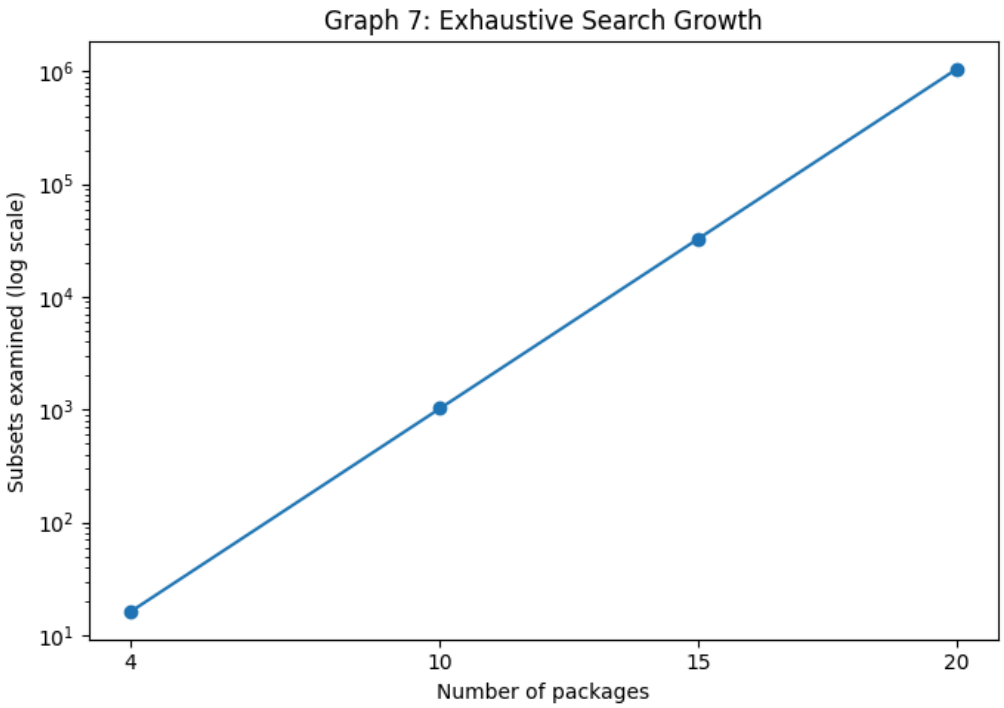

Graph 2: Sequential Search
First item
Middle item
Last item
Missing item
Comparisons (log scale)
10^3
10^2
10^1
10^0
10^1
10^2
10^3
List size (log scale)

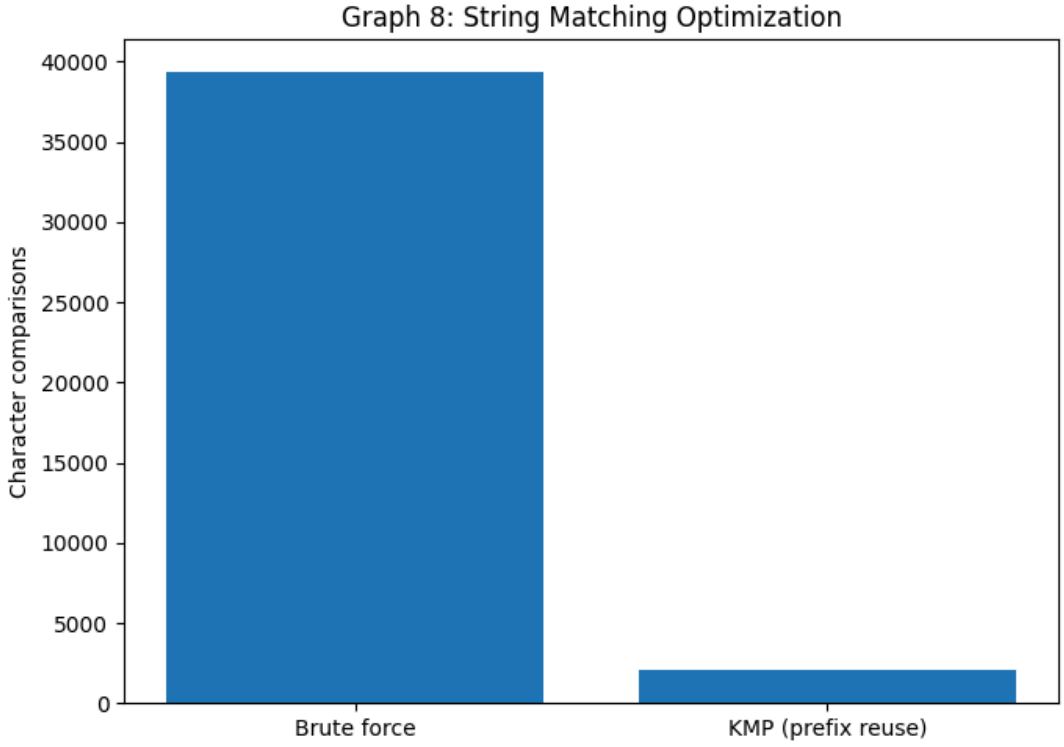

Graph 3: Brute-Force String Matching
Character comparisons
0
100
200
300
400
Low work
Repeated prefix
Match near end


Graph 4: Closest-Pair Operation Growth
Pairs examined
10^2
10^3
10^4
10^5
10^1
10^2
10^3
Number of drones


Graph 5: Convex Hull - Boundary and Interior
Input points
Convex hull
Y coordinate
X coordinate
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0


Graph 6: Convex Hull - Challenge Case
Input points
Convex hull
Y coordinate
X coordinate
3
2
1
0
−1
−2
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0


Graph 7: Exhaustive Search Growth
Subsets examined (log scale)
10^6
10^5
10^4
10^3
10^2
10^1
4
10
15
20
Number of packages


Graph 8: String Matching Optimization
Character comparisons
40000
35000
30000
25000
20000
15000
10000
5000
0
Brute force
KMP (prefix reuse)